

KARNATAKA RESIDENTIAL EDUCATIONAL INSTITUTIONS SOCIETY

ICT Class 10 - Practical Lab Exam 2

Practical Lab Exam 2 - Answer Key

Class: Class 10

Assessment Type: Practical Lab Exam 2

Total Marks: 20

Duration: 45 minutes

Topics Covered: SQL Database and Python with Database

ICT Teaching Resources Portal

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Lab Tasks (15 marks)

Task 1: SQL Queries (5 marks)

Expected Solution:

```
-- (a) Books with price above 500 sorted by title
SELECT * FROM Books WHERE price > 500 ORDER BY title ASC;

-- (b) Customers from Bangalore or Mysore
SELECT * FROM Customers WHERE city IN ('Bangalore', 'Mysore');

-- (c) Count books in each genre
SELECT genre, COUNT(*) AS book_count FROM Books GROUP BY genre;

-- (d) Total revenue for each book
SELECT b.title, SUM(b.price * s.quantity) AS total_revenue
FROM Books b JOIN Sales s ON b.book_id = s.book_id
GROUP BY b.book_id, b.title;

-- (e) Top 3 selling books by quantity
SELECT b.title, SUM(s.quantity) AS total_sold
FROM Books b JOIN Sales s ON b.book_id = s.book_id
GROUP BY b.book_id, b.title
ORDER BY total_sold DESC
LIMIT 3;

-- (f) Customers who haven't purchased
SELECT * FROM Customers
WHERE cust_id NOT IN (SELECT DISTINCT cust_id FROM Sales);

-- (g) Monthly sales summary
SELECT MONTH(sale_date) AS month, SUM(price * quantity) AS total_sales, SUM(quantity)
AS total_books
FROM Sales JOIN Books ON Sales.book_id = Books.book_id
GROUP BY MONTH(sale_date);
```

Marking Criteria:

- Each query: 0.7 marks (7 queries × 0.7 = 5 marks)

Task 2: Python with SQLite Database (5 marks)

Expected Solution:

```
import sqlite3

try:
    conn = sqlite3.connect('result.db')
    c = conn.cursor()
```

```

# Create tables
c.execute('''CREATE TABLE IF NOT EXISTS Students
            (roll INTEGER PRIMARY KEY, name TEXT, class TEXT)''')
c.execute('''CREATE TABLE IF NOT EXISTS Subjects
            (sub_id INTEGER PRIMARY KEY, sub_name TEXT, max_marks INTEGER)''')
c.execute('''CREATE TABLE IF NOT EXISTS Marks
            (roll INTEGER, sub_id INTEGER, marks INTEGER,
             FOREIGN KEY(roll) REFERENCES Students(roll),
             FOREIGN KEY(sub_id) REFERENCES Subjects(sub_id))''')

# Insert subjects
subjects = [(1, 'Math', 100), (2, 'Science', 100), (3, 'English', 100), (4,
'Kannada', 100)]
c.executemany('INSERT OR IGNORE INTO Subjects VALUES (?,?,?)', subjects)

# Insert students and marks
students = [(1, 'Rahul', '10A'), (2, 'Priya', '10A'), (3, 'Amit', '10B'),
            (4, 'Sneha', '10B'), (5, 'Vijay', '10A'), (6, 'Neha', '10A'),
            (7, 'Ravi', '10B'), (8, 'Pooja', '10B')]
c.executemany('INSERT OR IGNORE INTO Students VALUES (?,?,?)', students)

marks_data = [
    (1, 1, 85), (1, 2, 90), (1, 3, 78), (1, 4, 82),
    (2, 1, 92), (2, 2, 88), (2, 3, 95), (2, 4, 90),
    (3, 1, 78), (3, 2, 72), (3, 3, 80), (3, 4, 75),
    (4, 1, 88), (4, 2, 85), (4, 3, 82), (4, 4, 88),
    (5, 1, 71), (5, 2, 68), (5, 3, 75), (5, 4, 70),
    (6, 1, 95), (6, 2, 92), (6, 3, 98), (6, 4, 95),
    (7, 1, 82), (7, 2, 78), (7, 3, 85), (7, 4, 80),
    (8, 1, 76), (8, 2, 72), (8, 3, 78), (8, 4, 75)
]
c.executemany('INSERT INTO Marks VALUES (?,?,?)', marks_data)

# 1. Display all students with marks
c.execute('''SELECT s.name, sub.sub_name, m.marks
            FROM Students s JOIN Marks m ON s.roll = m.roll
            JOIN Subjects sub ON m.sub_id = sub.sub_id
            ORDER BY s.roll, sub.sub_id''')
print("Student Marks:")
for row in c.fetchall():
    print(f" {row[0]} - {row[1]}: {row[2]}")

# 2. Total and percentage
c.execute('''SELECT s.name, SUM(m.marks) AS total,
            (SUM(m.marks) / 400.0) * 100 AS percentage
            FROM Students s JOIN Marks m ON s.roll = m.roll
            GROUP BY s.roll ORDER BY total DESC''')
print("\nTotals:")
for row in c.fetchall():
    print(f" {row[0]}: Total={row[1]}, Percentage={row[2]:.1f}%")

# 3. Class topper
c.execute('''SELECT s.name, SUM(m.marks) AS total
            FROM Students s JOIN Marks m ON s.roll = m.roll
            GROUP BY s.roll ORDER BY total DESC LIMIT 1''')

```

```

topper = c.fetchone()
print(f"\nTopper: {topper[0]} with {topper[1]} marks")

# 4. Subject-wise toppers
c.execute('''SELECT sub.sub_name, s.name, m.marks
            FROM Subjects sub JOIN Marks m ON sub.sub_id = m.sub_id
            JOIN Students s ON m.roll = s.roll
            WHERE m.marks = (SELECT MAX(marks) FROM Marks WHERE sub_id =
sub.sub_id)''')
print("\nSubject Toppers:")
for row in c.fetchall():
    print(f" {row[0]}: {row[1]} ({row[2]})")

# 5. Failed students (percentage < 40%)
c.execute('''SELECT s.name, (SUM(m.marks)/400.0)*100 AS pct
            FROM Students s JOIN Marks m ON s.roll = m.roll
            GROUP BY s.roll HAVING pct < 40''')
print("\nFailed Students:")
for row in c.fetchall():
    print(f" {row[0]}: {row[1]:.1f}%")

# 6. Rank list
c.execute('''SELECT RANK() OVER(ORDER BY SUM(m.marks) DESC) AS rank,
            s.name, SUM(m.marks) AS total
            FROM Students s JOIN Marks m ON s.roll = m.roll
            GROUP BY s.roll''')
print("\nRank List:")
for row in c.fetchall():
    print(f" Rank {row[0]}: {row[1]} ({row[2]})")

conn.commit()

except sqlite3.Error as e:
    print(f"Database error: {e}")
finally:
    if conn:
        conn.close()

```

Marking Criteria:

- Database and table creation: 1 mark
- Data insertion: 1 mark
- Query operations (display, calculate, topper): 1 mark
- Subject-wise toppers and failed students: 1 mark
- Rank list and exception handling: 1 mark

Task 3: Python - Data Analysis with CSV (5 marks)

Expected Solution:

```

import csv
import collections

def analyze_sales():
    try:
        with open('sales_data.csv', 'r') as f:
            reader = csv.DictReader(f)
            data = list(reader)

```

```

# 1. Total sales by region
region_sales = collections.defaultdict(float)
for row in data:
    region_sales[row['Region']] += int(row['Quantity']) * float(row['Price'])

# 2. Best-selling product
product_qty = collections.defaultdict(int)
for row in data:
    product_qty[row['Product']] += int(row['Quantity'])
best_product = max(product_qty, key=product_qty.get)

# 3. Daily sales trend
daily_sales = collections.defaultdict(float)
for row in data:
    daily_sales[row['Date']] += int(row['Quantity']) * float(row['Price'])

# 4. Region with highest average order value
region_orders = collections.defaultdict(list)
for row in data:
    region_orders[row['Region']].append(int(row['Quantity']) *
float(row['Price']))
avg_orders = {r: sum(v)/len(v) for r, v in region_orders.items()}
best_region = max(avg_orders, key=avg_orders.get)

# 5. Products with zero/negative stock (simulated)
low_stock = [row['Product'] for row in data if int(row['Quantity']) <= 0]

# Write report
with open('analysis_report.txt', 'w') as f:
    f.write("Sales Analysis Report\n")
    f.write("=" * 40 + "\n\n")
    f.write("1. Total Sales by Region:\n")
    for region, total in region_sales.items():
        f.write(f"    {region}: Rs. {total:.2f}\n")
    f.write(f"\n2. Best-Selling Product: {best_product}\n")
    f.write(f"\n3. Daily Sales:\n")
    for date, total in sorted(daily_sales.items()):
        f.write(f"    {date}: Rs. {total:.2f}\n")
    f.write(f"\n4. Highest Avg Order Value: {best_region} (Rs.
{avg_orders[best_region]:.2f})\n")
    f.write(f"\n5. Low Stock Products: {low_stock if low_stock else
'None'}\n")

# Display results
print("Analysis complete. Report written to analysis_report.txt")
print(f"\nRegion Sales: {dict(region_sales)}")
print(f"Best Product: {best_product}")
print(f"Best Region: {best_region}")

except FileNotFoundError:
    print("Error: sales_data.csv not found!")
except PermissionError:
    print("Error: No file permission!")
except Exception as e:
    print(f"Error: {str(e)}")

```

`analyze_sales()`

Marking Criteria:

- CSV reading: 1 mark
- Region sales and product analysis: 1 mark
- Daily trend and average calculation: 1 mark
- Report writing: 1 mark
- Exception handling: 1 mark

Viva Voce (5 marks)

1. Primary key and composite primary key

- **Answer:** Primary key uniquely identifies each row. A table can have composite primary key (multiple columns combined). Example: `PRIMARY KEY(roll, sub_id)` in Marks table - individual values may repeat but combination is unique.
- **Marking:** 1 mark for explanation.

2. INNER JOIN vs LEFT JOIN

- **Answer:** INNER JOIN returns only matching rows from both tables. LEFT JOIN returns all rows from left table and matching from right (NULLs for non-matches). Use INNER when you need only matched data; use LEFT to keep all records from left table.
- **Marking:** 1 mark for difference.

3. GROUP BY vs ORDER BY

- **Answer:** GROUP BY groups rows with same values for aggregate functions (COUNT, SUM, AVG). ORDER BY sorts the result set by specified column(s). GROUP BY is for aggregation; ORDER BY is for sorting output.
- **Marking:** 1 mark for explanation.

4. sqlite3 module and commit()

- **Answer:** `sqlite3.connect()` creates connection, `cursor()` creates cursor for executing queries. `commit()` saves changes to database. Without `commit()`, changes are lost when connection closes (they exist only in memory).
- **Marking:** 1 mark for explanation.

5. Normalization 1NF and 2NF

- **Answer:** 1NF: Each column contains atomic values, no repeating groups. Violation: phone numbers in comma-separated string. 2NF: 1NF + no partial dependencies (all non-key columns depend on entire primary key). Violation: student name depends only on roll, not on (roll, subject) composite key.
- **Marking:** 1 mark for each NF with example.